Lecture 11 — Number Theory II: Modular Arithmetic, Fast Power, Inverses, nCr mod p

> Year Curriculum · Prerequisites: Week 9 (binary representation), Week 10 (divisibility, gcd, primes) · Estimated time: 2 × 90 min

1. Overview

Counting problems have answers that grow exponentially: the number of ways to arrange 10^6 items has millions of digits. Rather than ask for big-integer output, problem setters ask for the answer **modulo** a prime, almost always $10^9 + 7$ or 998244353. This lecture teaches you to do arithmetic in that world correctly:

- the rules for +, − and × modulo m, and the overflow and negative-number traps;
- binary exponentiation, computing $a^b \bmod m$ in O(log b);
- modular inverses via Fermat's little theorem, which let you *divide*;
- precomputed factorials and inverse factorials, giving binomial coefficients in O(1);
- a short catalogue of counting formulas (stars and bars, multinomials, Catalan numbers) that reduce to factorial ratios.

Combinatorics returns in depth in Week 30 (inclusion–exclusion, probability and expected value — expected values are also printed "mod p"), and matrix exponentiation in the Nationals track is binary exponentiation applied to matrices. The extended Euclidean algorithm, which computes inverses for non-prime moduli, is covered in the Regionals track.

2. Arithmetic modulo m

For an integer x and modulus $m > 0$, $x \bmod m$ is the remainder in $[0, m)$. Two numbers are **congruent**, $a \equiv b \pmod m$, if m divides $a - b$. Congruence respects addition, subtraction and multiplication:

- $(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$
- $(a - b) \bmod m = ((a \bmod m) - (b \bmod m)) \bmod m$, taken back into $[0, m)$
- $(a \cdot b) \bmod m = ((a \bmod m) \cdot (b \bmod m)) \bmod m$

So you can reduce intermediate results at any time without changing the final answer. That is what makes "answer modulo p" work: compute the whole thing in reduced form.

Division is different. (6 / 2) mod 4 = 3, but (6 mod 4) / (2 mod 4) = 2 / 2 = 1. To divide, we need inverses (section 4).

2.1 Overflow

With $m = 10^9 + 7 < 2^{30}$, two reduced values are $< 2^{30}$, their sum is $< 2^{31}$ (dangerously close to `INT_MAX`), and their product is $< 2^{60}$, which fits in a signed 64-bit integer (max $\approx 9.2 \cdot 10^{18}$). So:

- store values as `long long` (or as `int` but cast to `long long` before multiplying);
- reduce after **every** multiplication: `a * b % MOD * c % MOD`, never `a * b * c % MOD`;
- if the modulus itself is up to 10^{18}, a product of two residues needs 120 bits: use `__int128` for the multiplication, `(long long)((__int128)a * b % m)`.

2.2 Negative numbers

In C++, `%` truncates toward zero, so the result has the sign of the dividend: `-7 % 5 == -2`, not 3. After a subtraction, normalise:

```
x = ((x - y) % MOD + MOD) % MOD;     // safe for any sign of x - y
x = (x - y + MOD) % MOD;             // enough when 0 <= x, y < MOD
```

A negative residue that slips through will usually not crash; it just produces a wrong answer that matches the samples half the time.

3. Binary exponentiation

Computing a^b by $b - 1$ multiplications is hopeless for $b = 10^{18}$. Instead write b in binary. If $b = 13 = 1101_2$ then $a^{13} = a^8 \cdot a^4 \cdot a^1$. The powers a, a^2, a^4, a^8, … are obtained by repeated squaring, and we multiply into the result exactly those whose bit is set in b.

```
long long power(long long a, long long b, long long m) {
    long long r = 1 % m;      // "1 % m" handles m == 1
    a %= m; if (a < 0) a += m;
    while (b > 0) {
        if (b & 1) r = r * a % m;
        a = a * a % m;
        b >>= 1;
    }
    return r;
}
```

Trace for 3^{13} mod 1000:

step	b	bit	r after	a after
1	1101_2	1	3	9
2	110_2	0	3	81
3	11_2	1	243	561 (6561 mod 1000)
4	1_2	1	323 (243 · 561 = 136,323)	—

Indeed 3^{13} = 1,594,323. The loop runs once per bit of b, so the cost is **O(log b)** multiplications: 60 for b = 10^{18}. The same idea works for anything with an associative multiplication — matrices (linear recurrences), permutations (apply a permutation k times), and so on.

A recursive version, a^b = (a^(b/2))² · (a if b is odd), is equivalent; the iterative one avoids recursion and is the one to memorise.

4. Modular inverses

An **inverse** of a modulo m is a number x with a · x ≡ 1 (mod m). If it exists, dividing by a is the same as multiplying by x. Example modulo 7: 3 · 5 = 15 ≡ 1, so $3^{-1} \equiv 5$, and "4 / 3" ≡ 4 · 5 = 20 ≡ 6. Check: 3 · 6 = 18 ≡ 4. ✓

The inverse exists **if and only if gcd(a, m) = 1**. When m is prime, that is every a not divisible by m — the main reason contest moduli are prime.

4.1 Fermat's little theorem

For a prime p and an integer a not divisible by p, a^(p−1) ≡ 1 (mod p). Multiply out one factor of a: a · a^(p−2) ≡ 1, so

$a^{-1} \equiv$ a^(p−2) (mod p).

With binary exponentiation this is about 30 multiplications for p = 10^9 + 7. For instance $2^{-1} \equiv$ 500000004, and 5/6 ≡ 833333340 (mod 10^9 + 7).

Fermat's theorem has a second, equally useful consequence: since a^(p−1) ≡ 1, the value of a^e mod p only depends on e mod (p − 1) (again when p ∤ a). This is how CSES 1712 *Exponentiation II*, a^(b^c) mod p, is solved: compute e = b^c mod (p − 1) with fast power, then a^e mod p. The classic bug is reducing the exponent mod p instead of p − 1. If p | a the reduction is invalid; the answer is then 0 unless the true exponent b^c is 0.

For a non-prime modulus with gcd(a, m) = 1, use the extended Euclidean algorithm (or Euler's theorem a^φ(m) ≡ 1). Both are part of the Regionals number theory lecture.

4.2 All inverses 1..n in linear time

Write $p = q \cdot i + r$ with $q = \lfloor p / i \rfloor$ and $r = p \bmod i$. Then $q \cdot i + r \equiv 0$, so $i \equiv -r \cdot q^{-1}$ and $i^{-1} \equiv -q \cdot r^{-1}$. Since $r < i$, its inverse is already known:

```
inv[1] = 1;
for (int i = 2; i <= n; i++) inv[i] = (MOD - (MOD / i) * inv[MOD % i] % MOD) % MOD;
```

This gives n inverses in O(n) instead of O(n log p).

5. Binomial coefficients modulo p

C(n, k) counts k-element subsets of an n-element set and equals n! / (k! (n − k)!).

5.1 Factorials and inverse factorials

Precompute `fact[i] = i! mod p` for $i \le N$. For the inverses, compute only `invFact[N] = fact[N]^(p−2)` with one exponentiation and walk downwards using $1/(i-1)! = i \cdot (1/i!)$:

```
fact[0] = 1;
for (int i = 1; i <= N; i++) fact[i] = fact[i - 1] * i % MOD;
invFact[N] = power(fact[N], MOD - 2, MOD);
for (int i = N; i > 0; i--) invFact[i - 1] = invFact[i] * i % MOD;
```

Then C(n, k) = fact[n] · invFact[k] · invFact[n − k], reducing after each product, and returning 0 when $k < 0$ or $k > n$. Setup is O(N + log p), each query O(1). For CSES 1079 ($N = 10^6$, 10^5 queries) this runs in a few milliseconds.

This requires $N < p$: if $n \ge p$ then $n! \equiv 0 \pmod p$ and the formula breaks. With $p \approx 10^9$ that never happens in practice; with small primes use Lucas' theorem.

5.2 Pascal's triangle

$C(n, k) = C(n-1, k-1) + C(n-1, k)$ builds a table in $O(N^2)$. It needs no inverses, so it works for **any** modulus, including composite ones like 10^9 or 2^{64} (plain unsigned overflow). Without any modulus, all C(n, k) with $n \le 66$ fit in a signed 64-bit integer. Use Pascal when $N \le$ a few thousand and either the modulus is not prime or you want to avoid inverses altogether.

5.3 When n is huge but k is small

If $n \le 10^9$ but $k \le 10^6$, compute $C(n, k) = n (n-1) \dots (n-k+1) / k!$ directly: k multiplications for the numerator and one inverse of k!. AtCoder ABC 156 D *Bouquet* needs exactly this ($2^n - 1 - C(n, a) - C(n, b)$ with n up to 10^9).

6. A counting toolbox

Most "count the ways" problems at this level reduce to one of these:

- **Lattice paths.** Paths from (0, 0) to (a, b) using unit steps right and up: choose which a of the a + b steps go right, C(a + b, a). AtCoder ABC 145 D *Knight* transforms knight moves into this form.
- **Stars and bars.** Ways to put m identical items into n distinct boxes (empty allowed): arrange m stars and n − 1 bars in a row, C(m + n − 1, n − 1). Worked example (CSES 1716 *Distributing Apples*): n = 3 children, m = 2 apples gives C(4, 2) = 6 — namely (2,0,0), (0,2,0), (0,0,2), (1,1,0), (1,0,1), (0,1,1). If each box must be non-empty, give one item to each first: C(m − 1, n − 1).
- **Multinomials.** Distinct arrangements of a multiset with counts c_1, …, c_k summing to n: n! / (c_1! c_2! … c_k!). For CSES 1715 *Creating Strings II*, "aabac" has 5!/(3! 1! 1!) = 20 arrangements.
- **Catalan numbers.** Balanced bracket sequences with n pairs: C_n = C(2n, n) / (n + 1), computed as C(2n, n) · inv(n + 1). C_3 = 5: `((()))`, `(()())`, `(())()`, `()(())`, `()()()`. CSES 2064 *Bracket Sequences I*. Catalan numbers also count binary trees, triangulations and monotone lattice paths staying under the diagonal.
- **Powers.** Strings of length n over an alphabet of size k: k^n. Subsets: 2^n. Both via fast power.

7. A modular-integer type

Writing `% MOD` by hand everywhere invites mistakes. Many competitors keep a small struct in their notebook that overloads the operators; then `Mint` code reads like ordinary arithmetic. A full version is in section 8.3. Test it once carefully (negative inputs, division, exponent 0) and reuse it.

8. Reference implementations

8.1 CSES 1079 — Binomial Coefficients

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
const ll MOD = 1'000'000'007;

ll power(ll b, ll e, ll m) {          // b^e mod m, O(log e)
    ll r = 1 % m;
    b %= m; if (b < 0) b += m;
    while (e > 0) {
        if (e & 1) r = r * b % m;
        b = b * b % m;
```

```
        e >>= 1;
    }
    return r;
}
ll inv(ll a) { return power(a, MOD - 2, MOD); }   // MOD prime, a not divisible by MOD

const int N = 1'000'000;
ll fact[N + 1], invFact[N + 1];
void precompute() {
    fact[0] = 1;
    for (int i = 1; i <= N; i++) fact[i] = fact[i - 1] * i % MOD;
    invFact[N] = inv(fact[N]);
    for (int i = N; i > 0; i--) invFact[i - 1] = invFact[i] * i % MOD;
}
ll nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invFact[r] % MOD * invFact[n - r] % MOD;
}

int main() {
    // CSES 1079 - Binomial Coefficients: n queries of C(a, b) mod 1e9+7, a,b <= 1e6
    ios::sync_with_stdio(false); cin.tie(nullptr);
    precompute();
    int n; cin >> n;
    while (n--) {
        int a, b; cin >> a >> b;
        cout << nCr(a, b) << '\n';
    }
}
```

Sample: `5 3 → 10`, `2 1 → 2`, `10 5 → 252`. The same program solves CSES 1716 by querying C(m + n − 1, n − 1) (size the tables for $2 \cdot 10^6$).

8.2 CSES 1712 — Exponentiation II

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// CSES 1712 - Exponentiation II: a^(b^c) mod p, p = 1e9+7.
// Fermat: if p does not divide a, a^(p-1) = 1 (mod p), so the exponent
// can be reduced modulo p-1 (NOT modulo p!).
const ll MOD = 1'000'000'007;
ll power(ll b, ll e, ll m) {
    ll r = 1 % m; b %= m;
    while (e > 0) { if (e & 1) r = r * b % m; b = b * b % m; e >>= 1; }
    return r;
}
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n; cin >> n;
    while (n--) {
        ll a, b, c; cin >> a >> b >> c;
```

```
        if (a % MOD == 0) {                         // Fermat does not apply
            bool expIsZero = (b == 0 && c > 0);     // true exponent b^c == 0
            cout << (expIsZero ? 1 : 0) << '\n';
            continue;
        }
        ll e = power(b, c, MOD - 1);
        cout << power(a, e, MOD) << '\n';
    }
}
```

Note the convention $0^0 = 1$, which `power` also follows (b = 0, c = 0 gives exponent 1).

8.3 Modular integer struct and linear inverses

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// A small modular-integer type: operators do the "% MOD" for you.
const int MOD = 1'000'000'007;
struct Mint {
    int v;
    Mint(ll x = 0) { x %= MOD; if (x < 0) x += MOD; v = (int)x; }
    Mint& operator+=(Mint o) { v += o.v; if (v >= MOD) v -= MOD; return *this; }
    Mint& operator-=(Mint o) { v -= o.v; if (v < 0) v += MOD; return *this; }
    Mint& operator*=(Mint o) { v = (int)((ll)v * o.v % MOD); return *this; }
    friend Mint operator+(Mint a, Mint b) { return a += b; }
    friend Mint operator-(Mint a, Mint b) { return a -= b; }
    friend Mint operator*(Mint a, Mint b) { return a *= b; }
    Mint pow(ll e) const {
        Mint r = 1, b = *this;
        for (; e > 0; e >>= 1, b *= b) if (e & 1) r *= b;
        return r;
    }
    Mint inv() const { return pow(MOD - 2); }        // MOD is prime, v != 0
    Mint& operator/=(Mint o) { return *this *= o.inv(); }
    friend Mint operator/(Mint a, Mint b) { return a /= b; }
};

int main() {
    // Inverses of 1..n in O(n): inv[i] = -(MOD / i) * inv[MOD % i]
    int n = 10;
    vector<Mint> inv(n + 1);
    inv[1] = 1;
    for (int i = 2; i <= n; i++) inv[i] = Mint(0) - Mint(MOD / i) * inv[MOD % i];
    for (int i = 1; i <= n; i++) assert((inv[i] * i).v == 1);
    cout << "inv[2] = " << inv[2].v << "\n";                          // 500000004
    cout << "(1/2 + 1/3) = " << (Mint(1) / 2 + Mint(1) / 3).v << "\n";  // 5/6 mod p
    cout << "5/6 check: " << (Mint(5) / 6).v << "\n";
    cout << "(-7) mod p = " << Mint(-7).v << "\n";
    cout << "2^(10^18) = " << Mint(2).pow(100000000000000000LL).v << "\n";
}
```

Output: `500000004`, `833333340` twice (1/2 + 1/3 = 5/6), `1000000000`, `719476260`.

9. Common mistakes

- **Overflow before the modulus.** `int a, b; long long x = a * b % MOD;` multiplies in `int` first. Write `1LL * a * b % MOD`.
- **Chained products.** `a * b * c % MOD` overflows for residues near 10^9; reduce after each multiplication.
- **Negative residues** after subtraction (`(a - b) % MOD` can be negative). Add MOD.
- **Unreduced input to `power`.** If $a \geq MOD$ or $a < 0$, the first `a * a` can overflow or go negative; reduce the base first.
- **Fermat with a composite modulus**, or inverting 0 (which silently returns 0).
- **Reducing exponents mod p** instead of mod $p - 1$.
- **Using a mod-reduced value in a comparison.** After reduction, "which is bigger" is meaningless; e.g. you cannot take the max of two answers mod p.
- **Factorial tables too small.** Stars and bars needs $n + m - 1$, Catalan needs $2n$. Size the tables for the largest argument actually used.
- **Wrong modulus constant.** Check whether it is $10^9 + 7$ or 998244353; a typo in one digit fails every non-trivial test.

10. Practice problems

Warm-up:

1. **CSES 1095 — Exponentiation** — https://cses.fi/problemset/task/1095
2. **CSES 1079 — Binomial Coefficients** — https://cses.fi/problemset/task/1079
3. **CSES 1716 — Distributing Apples** — https://cses.fi/problemset/task/1716

Core:

4. **CSES 1712 — Exponentiation II** — https://cses.fi/problemset/task/1712
5. **CSES 1715 — Creating Strings II** — https://cses.fi/problemset/task/1715 (multinomial).
6. **CSES 2064 — Bracket Sequences I** — https://cses.fi/problemset/task/2064 (Catalan).
7. **AtCoder ABC 145 D — Knight** — https://atcoder.jp/contests/abc145/tasks/abc145_d (reduce to lattice paths).
8. **AtCoder ABC 156 D — Bouquet** — https://atcoder.jp/contests/abc156/tasks/abc156_d (huge n, small k).

Challenge:

9. **Codeforces 300C — Beautiful Numbers** — https://codeforces.com/problemset/problem/300/C (iterate over the count of one digit, nCr per candidate).

10. **CSES 1082 — Sum of Divisors** — https://cses.fi/problemset/task/1082 (from Week 10; now you have the modular tools; careful with the inverse of 2).
11. **CSES 2182 — Divisor Analysis** — https://cses.fi/problemset/task/2182 (number, sum and product of divisors mod p; exponents mod $p - 1$).

11. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 21 "Number theory" (modular arithmetic, Fermat/Euler) and Chapter 22 "Combinatorics" (binomial coefficients, Catalan numbers): https://cses.fi/book/book.pdf
- CP-Algorithms: "Binary Exponentiation" https://cp-algorithms.com/algebra/binary-exp.html, "Modular Multiplicative Inverse" https://cp-algorithms.com/algebra/module-inverse.html, "Binomial Coefficients" https://cp-algorithms.com/combinatorics/binomial-coefficients.html, "Catalan numbers" https://cp-algorithms.com/combinatorics/catalan-numbers.html
- USACO Guide, Gold "Modular Arithmetic" and "Combinatorics": https://usaco.guide/gold/modular , https://usaco.guide/gold/combo
- Next steps: Week 30 (inclusion–exclusion, probability, expected value mod p); Regionals "Number theory" (extended Euclid, CRT, Euler φ); Nationals "Advanced math" (matrix exponentiation).